CE-355 Lab 3
Drum Pad Sampler

1. Introduction

FPGAs are well suited to real-time digital audio. They can read stored sound samples, combine several sounds at once, and stream the result to an audio codec at an exact, steady sample rate. In this lab, you will build a drum pad sampler. Four push buttons play kick, snare, hi-hat and crash drum sounds, a metronome plays a click at a selectable tempo, and the mixed audio is sent to the DE25-Standard's audio codec so you can hear it through an external stereo speaker.

This lab focuses on **clocked processes**: processes that update registers on the rising edge of the clock. You will use them to build **counters**, **sample-rate pulse generators**, **sequencers** that step through memory, and **pipelined arithmetic**. You will also build **ROM lookups** that Quartus maps into the FPGA's on-chip block memory.

You will again use **for-generate** statements, this time to create five identical drum voice players, one for each sound, and to pack and unpack multi-voice data buses.

Starter VHDL files are provided with **TODO sections** that you will complete. You will then simulate the design, listen to the simulated audio as a WAV file, and synthesize, program and test the completed system on the DE25-Standard.

By completing this lab, you will:

- Implement clocked processes with asynchronous reset.
- Build counters and clock-enable pulses that run logic at a slower rate than the system clock.
- Read sample data from ROMs inferred as FPGA block memory.
- Use for-generate statements to create parallel voice players and to pack and unpack buses.
- Mix multiple signed audio streams with saturation to prevent overflow.
- Simulate, program and test a real-time FPGA audio system.

2. Background: Digital Audio

A digital audio signal is a stream of numbers called **samples**. Each sample is the amplitude of the sound wave at one instant. When the samples are sent to a digital-to-analog converter at a steady **sample rate**, the original sound is reproduced. This lab uses **32,000 samples per second** (32 kHz), with each sample a **16-bit signed** value. This is the same format stored in a standard .wav file.

The drum pad works like a simple sampler:

1. Each drum sound is a one-second recording, stored on the FPGA in a ROM as a table of 32,000 samples.
2. Pressing a key starts reading that drum's table from the beginning, one sample per sample period, until the end of the recording.
3. Several drums can play at the same time, so the current samples of all the drums are added together into a single stream.
4. The mixed stream is sent to the board's audio codec, which converts it to an analog signal for the speaker.

All of this happens at the audio sample rate. The FPGA clock runs at 50 MHz, much faster than 32 kHz, so the design generates a **sample pulse** 32,000 times per second. Every module advances by one sample each time the pulse occurs.

3. Project Setup

You will use **ModelSim** to simulate and verify your designs, and **Altera Quartus Prime** to compile, synthesize and program your design on the DE25-Standard FPGA development board.

Organize your project using the same directory structure as Lab 2:

```
Lab 3/
├── vhdl/  — Contains the VHDL source files for the design.
├── sim/   — Contains the ModelSim command file and simulation output.
└── syn/   — Contains the Altera Quartus Prime synthesis files.
```

The provided VHDL files include TODO sections that you will complete during the lab. The design files are:

- audio_package.vhd — Provided. Sample ROM constants for the five drum sounds.
- audio_pulse.vhd — Sample-rate pulse generator.
- drum_voice.vhd — Steps through one sample ROM when triggered.
- metronome.vhd — Provided. Triggers the click sound and LED at a selectable.
- audio_mixer.vhd — Sums and saturates the five voices.
- drum_pad.vhd — Connects the pulse generator, metronome, voices, ROMs and mixer.
- audio_codec.vhd — Provided. Configures and streams audio to the SSM2603 codec.
- drum_pad_system.vhd — System-level architecture connecting drum_pad to audio_codec and the board pins.
- drum_pad_tb.vhd — Simulates drum_pad and writes the audio to a WAV file.

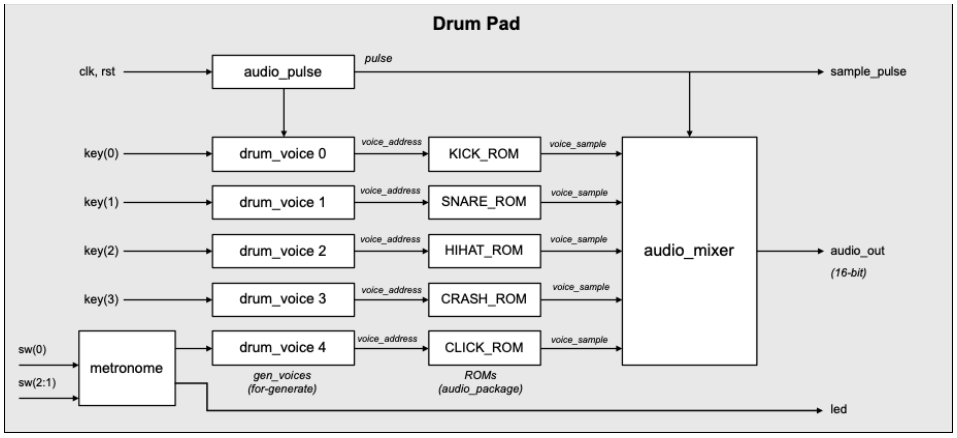
*NOTE: Throughout this assignment, please refer to the **DE25-Standard Development Kit User Manual** and the **DE25-Standard Getting Started Guide**, which contain more details on the board setup, pinouts, audio codec and configuration.*

4. System Design

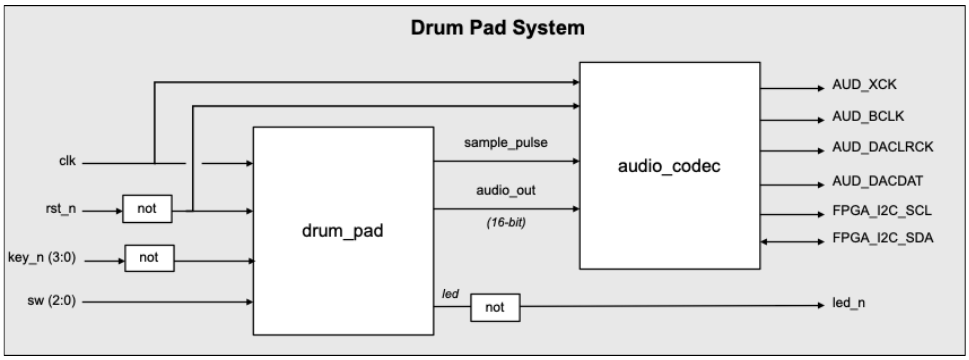
The drum pad sampler is built from several modules that share the 50 MHz clock and the sample pulse. The system has two levels of hierarchy.

drum_pad contains all the audio logic. You will simulate it on its own:

- **Audio Pulse** — Generates the one-cycle sample pulse at 32 kHz. Every other module uses it as a clock enable.
- **Metronome** — When sw(0) is on, triggers the click voice at 60, 90, 120 or 180 beats per minute, selected by sw(2 downto 1). It flashes the LED on each beat.
- **Drum Voices** — Five instances of drum_voice, created with a for-generate statement. Voices 0–3 are triggered by key(0)–key(3), and voice 4 is triggered by the metronome. Each voice outputs a ROM address.
- **Sample ROMs** — A registered ROM lookup reads one sample per voice from the constant arrays in audio_package.
- **Audio Mixer** — Sums the five voice samples and saturates the result into a single 16-bit audio sample, updated on each sample pulse.



The **drum_pad_system** is the board-level system architecture. It connects drum_pad to audio_codec, and it inverts the active-low reset button and LED. The data path therefore runs: **key press → drum voice → ROM address → ROM sample → mixer → codec → audio out**.



5. Design Implementation

In this section, you will complete the provided VHDL files. Work through the modules from the bottom of the hierarchy up, so that each one can be checked before it is used.

Every clocked process in this lab follows the same template:

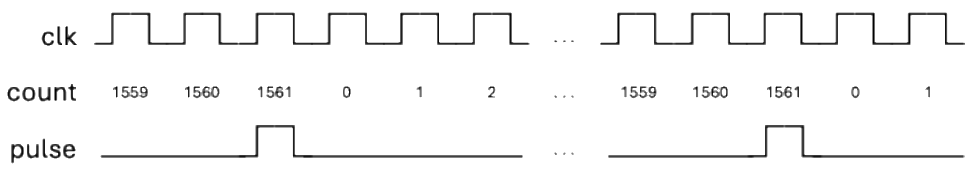
```
process_name : process(clk, rst)
begin
   if ( rst = '1' ) then
      -- asynchronous reset: give every register a known value
   elsif ( rising_edge(clk) ) then
      -- register updates on the rising clock edge
   end if;
end process process_name;
```

The sensitivity list contains only clk and rst. Signals read inside the rising_edge branch do **not** go in the sensitivity list, because the process only updates registers on a clock edge. Every signal assigned in the process becomes a register.

5.1 Implement the Audio Pulse Generator

The audio_pulse module generates the sample pulse: a signal that is high for exactly **one clock cycle** once per sample period. Rather than creating a second, slower clock, the whole design stays on the 50 MHz clock, and the other modules use the pulse as an **enable signal**. In other words, they check if pulse = '1' inside their clocked processes, then advance exactly one step per sample. The number of clock cycles per sample is:

AUDIO_DIVIDER = CLK_FREQUENCY / SAMPLE_RATE_HZ = 50,000,000 / 32,000 = 1562



Complete **audio_pulse_process** in **audio_pulse.vhd**. Declare a counter signal that counts clock cycles from 0 to AUDIO_DIVIDER-1 and then wraps back to 0. Assert pulse for exactly **one clock cycle** each time the counter wraps, and drive it to '0' on every other cycle. On reset, clear both the counter and pulse. Use a constrained integer for the counter, for example natural range 0 to AUDIO_DIVIDER-1, so that synthesis builds a counter only as wide as it needs to be.

5.2 Implement the Drum Voice

Complete **clocked_process** in **drum_voice.vhd**. A drum voice plays one sample from start to finish by stepping a ROM address through the sample. Declare signals for the current address and for a playing flag, and implement the following behavior:

- **Idle:** when playing is '0', the address stays at 0 and sample pulses are ignored.
- **Trigger:** on any clock where trigger is '1', reset the address to 0 and initiate the start of the audio by setting a playing signal to '1'. A trigger during playback restarts the sound from the beginning. The trigger has priority over the pulse.
- **Playing:** on each sample pulse while playing is '1', increment the address by one.
- **End:** on the pulse when the address reaches DATA_SIZE-1, return the address to 0 and clear playing.

Drive **rom_address** from the address register and active from the playing flag. Because these outputs come directly from registers, the ROM always receives a stable address.

5.3 Implement the Audio Mixer

Complete **audio_mixer.vhd**. The five voice samples arrive on a single flattened bus, voice_samples, which is NUM_VOICES × DATA_WIDTH bits wide, where each voice is a slice of DATA_WIDTH wide.

First, replace the placeholder assignment to voice_array with a **for-generate** statement that unpacks each 16-bit slice of voice_samples into one element of voice_array. Next, implement the mixer itself. Adding five 16-bit samples can produce a result that potentially may overflow in 16 bits, so the sum is computed with extra bits:

SUM_WIDTH = DATA_WIDTH + clog2(NUM_VOICES) = 16 + 3 = 19 bits

The 19-bit sum must then be converted back to 16 bits for the codec. If the upper bits are simply dropped, a large positive sum wraps around to a large negative value, which is heard as a loud click. Instead, the mixer **saturates** (clips) the result to the range AUDIO_MIN (−32,768) to AUDIO_MAX (+32,767).

At 50 MHz, each register-to-register path must settle within 20 ns, and adding five numbers and then comparing the result in the same clock cycle can take longer than that. The mixer therefore splits the work into two **pipeline stages**, each with a full clock period. Because the voice samples change only once every 1562 clocks, the extra clock of latency has no audible effect.

Implement the two pipeline stages in mixer_process:

- **Stage 1 (every clock):** add all five voices, converting each to signed and resizing it to SUM_WIDTH bits before adding, and register the result in mix_sum. A for loop with a variable accumulator keeps this compact.

- **Stage 2 (on the sample pulse):** when pulse = '1', saturate mix_sum to within AUDIO_MIN and AUDIO_MAX, then resize to DATA_WIDTH bits. Register the result in audio. When pulse = '0', audio holds its value.

5.4 Implement the Drum Pad

Complete the structural design in **drum_pad.vhd** as a structural design.

First, add the signal declarations and component declarations for **audio_pulse**, **metronome**, **drum_voice** and **audio_mixer**. You will need signals for the sample pulse, the metronome trigger, the five voice triggers, the five voice addresses (address_array_t), the ROM outputs and voice samples (sample_array_t), the flattened sample bus, and the mixed audio.

Then, connect the modules:

- **Audio pulse:** instantiate audio_pulse, passing CLK_FREQUENCY and SAMPLE_RATE_HZ through its generic map.
- **Metronome:** instantiate metronome. Connect sw(0) to enable, sw(2 downto 1) to speed, and the led output to the top-level led.
- **Voice triggers:** use concurrent assignments to drive the voice triggers. The push buttons are **active-low** ('0' when pressed), so voice *i* (for *i* = 0 to 3) is triggered by not key_n(i), which should be negated at the system level. Voice 4 is triggered by the metronome.
- **Drum voices:** use a **for-generate** statement to create five drum_voice instances, one per voice. Set ADDR_WIDTH to ROM_ADDR_WIDTH and DATA_SIZE to ROM_DEPTH from audio_package.
- **ROM lookup:** the drum sounds are stored in audio_package.vhd as constant arrays (KICK_ROM, SNARE_ROM, HIHAT_ROM, CRASH_ROM and CLICK_ROM), each holding 32,000 16-bit samples. When a constant array is read inside a **clocked process**, Quartus recognizes it as a ROM and places it in the FPGA's **M20K block memory**. If it is read combinationally, Quartus builds it from thousands of logic cells instead. Complete rom_process: on each rising clock edge, read each ROM at its voice's address. For example:

  ```
  rom_sample(0) <= KICK_ROM(to_integer(unsigned(voice_address(0))));
  ```

  Then **register the ROM outputs a second time into voice_sample**. This process has **no reset**, so clk is the only signal in its sensitivity list. Without a reset, Quartus can infer M20K block ROMs, and the second register stage uses the M20K's built-in output register.
- **Sample bus:** use a **for-generate** statement to pack the five **voice_sample** values into the flattened bus, in the layout shown in Section 5.3.

- **Mixer:** instantiate **audio_mixer**, and drive sample_pulse and audio_out from the pulse and the mixed audio.

5.5 Implement the Top-Level Drum Pad System

Complete **drum_pad_system.vhd** as a structural design.

Note that the reset push button and the LED are both **active-low** on the board. Use concurrent assignments to invert them before supplying to the modules, which expect positive-edge reset.

Add the component declarations and instantiate **drum_pad** and **audio_codec**, which is provided for you. Pass the same DATA_WIDTH, CLK_FREQUENCY and SAMPLE_RATE_HZ generics to both, connect the drum pad's sample_pulse and audio_out to the codec's pulse and audio inputs, and connect the codec's audio and I2C ports to the top-level ports.

6. Simulation

Before programming the FPGA, verify the drum pad using ModelSim. The testbench simulates **drum_pad** (not drum_pad_system), so the codec is not simulated. Instead, the testbench captures audio_out on every sample pulse and writes it to **drum_pad.wav** in the sim directory. You can play this file on your computer to **hear** your simulation.

6.1 Testbench Setup

The provided **drum_pad_tb** already instantiates the DUT, generates the clock and reset, and writes the WAV file. Read through it before you begin.

To keep simulation time reasonable, the testbench runs the clock at **320 kHz** instead of 50 MHz (CLK_PERIOD_NS = 3125 ns). The testbench passes the matching CLK_FREQUENCY to the DUT, so AUDIO_DIVIDER becomes 10 clocks per sample instead of 1562. The audio output is identical, but the simulation runs about 156 times faster. Synthesis still uses the 50 MHz default.

6.2 Test Stimulus

The provided stimulus enables the metronome at its fastest tempo (sw <= "111"), lets it run for 4 seconds, and then presses key(0) once. Complete the TODO in **stimulus_process**:

- Press and release each of the remaining keys, key(1) through key(3), one at a time. Remember that the keys are active-low, but they are inverted at the system-level. After each press, wait at least 1 second so the full sound is heard before the next key.
- Press several keys at the same time to test **polyphony** and the mixer's saturation.
- Optionally, change sw to test the other metronome tempos, or turn the metronome off.

Each additional second of stimulus adds to the simulation run time. Keep the total test to around 10 seconds of simulated audio.

6.3 Create the ModelSim Command File

Complete the file sim.do in the sim directory. It should compile the VHDL files in bottom-up order, starting with audio_package, start the drum_pad_tb testbench, add the important signals to the waveform viewer, and run the simulation:

```
vlib work

vcom -2008 ../vhdl/audio_package.vhd
vcom -2008 ../vhdl/metronome.vhd
vcom -2008 ../vhdl/audio_pulse.vhd
vcom -2008 ../vhdl/audio_mixer.vhd
vcom -2008 ../vhdl/drum_voice.vhd
vcom -2008 ../vhdl/drum_pad.vhd
vcom -2008 ../vhdl/drum_pad_tb.vhd

vsim -voptargs=+acc drum_pad_tb

add wave -group "TESTBENCH" sim:/drum_pad_tb/*
add wave -group "DRUM PAD" sim:/drum_pad_tb/dut/*
add wave -group "AUDIO PULSE" sim:/drum_pad_tb/dut/audio_pulse_inst/*
add wave -group "AUDIO MIXER" sim:/drum_pad_tb/dut/audio_mixer_inst/*
add wave -group "METRONOME" sim:/drum_pad_tb/dut/metronome_inst/*
add wave -group "KICK VOICE" sim:/drum_pad_tb/dut/gen_voices(0)/voice_inst/*

run -all
```

Adjust the instance names (audio_pulse_inst, gen_voices, voice_inst and so on) to match the labels you used in drum_pad.vhd. The run -all command runs until the testbench finishes writing the WAV file and stops the simulation.

Note that the audio_package.vhd contains five 32,000-entry ROMs, so it may take noticeably longer to compile than the other files.

6.4 Run the Simulation

Open ModelSim and change to the sim directory. Run the command file by entering:

```
do sim.do
```

Alternatively, the simulation can be launched directly from a terminal:

```
vsim -do sim.do
```

6.5 Verify the Waveforms and Audio

In the waveform viewer, verify that:

- pulse is high for exactly one clock cycle every 10 clocks (in simulation).
- After a key press, the corresponding voice's playing flag goes high, its address counts up once per pulse, and it stops after 32,000 samples.
- The metronome triggers the click voice once per beat, and led flashes on each beat.
- mix_sum and audio_out change when voices are playing, and audio_out is clipped at +32,767 / −32,768 rather than wrapping when several loud voices overlap.
- Reset returns all counters and outputs to zero.

Set the radix of audio_out to **decimal** and its format to **analog** (right-click → *Format* → *Analog*) to view it as a waveform. Finally, open sim/drum_pad.wav and **listen** to it. You should hear the metronome clicks, followed by each drum sound in turn, then the drums playing together. Crackling or loud pops usually mean the mixer is wrapping instead of saturating, or that the samples are being treated as unsigned.

7. Synthesis

Once the design has been simulated, implement the drum pad sampler on the DE25-Standard.

7.1 Create the Quartus Prime Project

Create a new Quartus Prime project in the syn directory. Name the project drum_pad and set **drum_pad_system** as the **top-level entity**. Configure the project for the Agilex 5 SoC FPGA used on the DE25-Standard, device **A5ED013BB32AE4SCS**. Add the following VHDL source files from the vhdl directory:

- audio_package.vhd
- audio_pulse.vhd
- drum_voice.vhd
- metronome.vhd
- audio_mixer.vhd
- drum_pad.vhd
- audio_codec.vhd
- drum_pad_system.vhd

Include the provided **drum_pad.sdc** in the syn directory with the following clock constraint, and add it to the project:

```
create_clock -name clk -period "50MHz" [get_ports clk]
```

7.2 Assign FPGA Pins

Open **Assignments → Pin Planner** and assign the following signals:

Port	FPGA Pin	I/O Standard	Description
clk	CH128	3.3-V LVCMOS	50 MHz FPGA clock
rst_n	BM78	1.2-V	CPU reset
key_n[0]	BW59	1.2-V	Push button 0 (kick)
key_n[1]	CA59	1.2-V	Push button 1 (snare)
key_n[2]	CF71	1.2-V	Push button 2 (hi-hat)
key_n[3]	CH71	1.2-V	Push button 3 (crash)
sw[0]	BM62	1.2-V	Slide switch 0 (metronome on/off)
sw[1]	BP62	1.2-V	Slide switch 1 (tempo)
sw[2]	BH62	1.2-V	Slide switch 2 (tempo)
led_n	CC71	1.2-V	LED 0 (metronome beat)
AUD_XCK	CK97	1.2-V	Codec master clock
AUD_BCLK	CL97	1.2-V	Codec bit clock
AUD_DACLRCK	CK94	1.2-V	Codec DAC left/right clock
AUD_DACDAT	CL88	1.2-V	Codec DAC serial data
FPGA_I2C_SCL	BF120	3.3-V LVCMOS	Codec I2C clock
FPGA_I2C_SDA	BH118	3.3-V LVCMOS	Codec I2C data

7.3 Compile and Synthesize

Run the complete Quartus Prime compilation flow. The compilation should complete without errors. Review the compilation reports and verify that:

- drum_pad_system is the top-level entity.
- All required VHDL files are included.
- All top-level ports have valid pin assignments and I/O standards.
- The five sample ROMs were inferred as **M20K block memory**. Check the **Fitter → Resource Usage Summary** for M20K blocks. If the ROMs were built from LUTs instead, the logic utilization will be very high, which usually means the ROM read is not registered or has a reset.

- No unexpected unconnected or undriven signals are reported.
- Timing analysis meets the 50 MHz constraint with no setup or hold violations. Check the **Timing Analyzer** slack for clk.

8. Hardware Setup and Testing

Make all connections with the FPGA board powered off. Connect the DE25-Standard to the computer through the board's **USB-Blaster** connection.

Connect the provided external stereo speaker to the DE25-Standard's **audio-out** jack using the 3.5 mm audio cable. Power on the speaker. The design sends the same mono signal to both the left and right channels, so both sides of the speaker should play the same sound.

Power on the DE25-Standard and open **Tools → Programmer** in Quartus Prime. Select the hardware connection and the .sof file, then program the FPGA.

After programming, the codec is configured automatically over I2C, which takes about 20 ms. Then test the system:

- Press each push button and verify that it plays the correct sound: key[0] kick, key[1] snare, key[2] hi-hat, key[3] crash.
- Press several buttons at once and verify that the sounds mix together without crackling.
- Press a button repeatedly and verify that each press restarts the sound.
- Turn on sw[0] and verify that the metronome clicks and LED 0 flashes on each beat.
- Change sw[2:1] and verify the four tempos: 60, 90, 120 and 180 BPM.
- Press the reset button and verify that all sound stops.

9. Submission

Zip your folder, name it **CE355_Lab3_Team_[N].zip**, and submit it to Canvas. Make sure to clean out any synthesis and simulation work directories. Do not include the generated .wav and .raw files.

You will also need to demo the hardware setup and simulation during TA office hours. Be prepared to explain how the system works, including the sample pulse, the drum voice sequencing, the mixer pipeline and saturation, and how your simulation results compare with the hardware behavior.